

Лабораторная работа №42

Разработка веб-приложения для доступа к данным

1 Цель работы

- 1.1 Научиться создавать веб-приложения, используя технологию ASP.NET Razor Pages.
- 1.2 Научиться реализовывать работу с БД, используя технологию EF Core.

2 Литература

- 2.1 <https://metanit.com/sharp/razorpages/> гл.4

3 Подготовка к работе

- 3.1 Повторить теоретический материал (см. п.2).
- 3.2 Изучить описание лабораторной работы.

4 Основное оборудование

- 4.1 Персональный компьютер.

5 Задание

Создать в одном решении 2 проекта «Веб-приложение ASP.NET Core (Майкрософт)»:
- для задания 1,
- для заданий 2-5.

5.1 Выполнение учебного примера

Создать проект веб-приложения. Выполнить задания из гл.4 по ссылке <https://metanit.com/sharp/razorpages>. БД создавать не требуется, вместо таблицы User использовать существующую таблицу заказчиков (Customers), скорректировав описание свойств под структуру своей таблицы. Строку подключения настроить на подключение к своей БД.

5.2 Отображение списка книг

5.2.1 Создать проект веб-приложения и добавить в него:

- пакет Microsoft.EntityFrameworkCore.SqlServer,
- контекст для работы с БД,
- папку Models с классами модели данных для таблиц Книги и Авторы,
- строку подключения DefaultConnection в файле appsettings.json.

5.2.2 Зарегистрировать контекст данных как сервис в Program.cs.

5.2.3 На страницу Index.cshtml с моделью IndexModel внедрить ApplicationContext через конструктор модели страницы. В методе OnGetAsync() получить из контекста всех книг методом ToListAsync() и передать их в представление.

5.2.4 На страницу Index.cshtml вывести список книг в виде таблицы.

5.3 Отображение подробной информации и удаление записи

5.3.1 Создать страницу Details.cshtml, которая:

- через параметр маршрута id (с директивой @page "{id:int}") принимает идентификатор книги,
- находит книгу в БД (FindAsync(id)),
- загружает данные о книге в открытое свойство Book,

- отображает всю информацию о книге на странице.

Если книга не найдена, вернуть статус 404 (NotFound()).

5.3.2 Добавить в таблицу на странице Index.cshtml для каждой строки с книгой кнопку «Подробнее», при нажатии на которую перенаправлять на страницу Details.cshtml и отображать все данные о выбранной книге.

Для перехода можно использовать тэг-хелпер:

`<a asp-page="Details" asp-route-id="@book.Id">Подробнее</a>`

5.3.3 Добавить на страницу Details.cshtml кнопку «Удалить», при нажатии на которую в методе OnPostDeleteAsync(int id) удалять из БД данные о выбранной книге, используя Remove() и SaveChangesAsync().

Для передачи id в форме использовать: `<input type="hidden" asp-for="Book.Id" />`

В форме использовать обработчик `asp-page-handler="Delete"`

5.3.4 Добавить в таблицу на странице Index.cshtml для каждой строки с книгой кнопку «Удалить», при нажатии на которую удалять из БД данные о выбранной книге.

5.4 Создание новой записи

5.4.1 Создать страницу Create.cshtml. Реализовать переход к ней со страницы Index.cshtml при нажатии на кнопку «Добавить».

5.4.2 На странице Create.cshtml создать форму для ввода данных о книге, которая отправляет данные методом POST.

5.4.3 В модели страницы:

- определить публичные свойства для полей ввода с данными о книге с атрибутом [BindProperty] и связать их с интерфейсом,

- реализовать обработчик OnPostAsync(): если переданные данные валидны, создать новый объект Книга, добавить его в контекст через AddAsync() и сохранить изменения через SaveChangesAsync().

5.4.4 После успешного сохранения выполнить перенаправление на страницу Index.cshtml.

5.4.5 Добавить проверку на пустые поля. Обработать сценарий, когда при сохранении возникла ошибка, и вернуть книгу обратно на страницу создания с сообщением.

5.5 Редактирование существующей записи

5.5.1 Создать страницу Edit.cshtml с шаблоном маршрута `@page "{id:int}"`. Реализовать переход к ней со страницы Details.cshtml или Index.cshtml при нажатии на кнопку «Редактировать».

5.5.2 Реализовать обработчик OnGetAsync(int id), который находит книгу по id и загружает ее данные в публичные свойства с атрибутом [BindProperty].

5.5.3 На странице Edit.cshtml создать форму, где все поля формы уже заполнены текущими значениями книги.

5.5.4 Реализовать обработчик OnPostAsync():

- найти книгу через FindAsync(),
- обновить её свойства значениями из формы,
- сохранить изменения в БД через SaveChangesAsync(),
- выполнить перенаправление обратно на страницу Index.

5.5.5 Передавать id скрытым полем в форме. Обращать внимание на ситуацию, когда при OnGet книга не найдена (404).

6 Порядок выполнения работы

6.1 Запустить MS Visual Studio и создать на C# набор требуемых проектов.

6.2 Выполнить все задания из п.5. При выполнении заданий использовать минимально возможное количество команд и переменных и выполнять форматирование и рефакторинг кода.

6.3 Ответить на контрольные вопросы.

7 Содержание отчета

7.1 Титульный лист

7.2 Цель работы

7.3 Ответы на контрольные вопросы

7.4 Вывод

8 Контрольные вопросы

8.1 Какой метод используется для сохранения изменений в БД?

8.2 Как зарегистрировать контекст данных в Program.cs?

8.3 Какой атрибут нужен для привязки полей формы к открытым свойствам?

8.4 Где настраивается строка подключения к БД в веб-приложении?

8.5 Как вернуть статус 404, если запись не найдена?